

AZURE SITE RECOVERY

Three-Tier SpendWise Disaster Recovery Project

*Step-by-Step Implementation, Commands, Real Execution Outputs,
Concepts, Validation and Review Guide*

Primary Region	Central India
Recovery Region	India South Central
Recovery Services Vault	RSV-ASR-24CC3046
Protected VMs	VM-DB, VM-APP, VM-WEB
Recovery Plan	RP-3TIER-APP
Test VNet	VNET-ASR-TEST / SUBNET-TEST

Evidence basis: This document uses the commands and outputs captured during the actual project execution in this conversation. Outputs are reproduced/condensed where they are very long.

Important status: Azure Site Recovery protection, recovery plan creation, test failover, DR VM creation, network placement, service connectivity and application health validation were completed successfully. Test-failover cleanup was attempted through Azure CLI but the installed CLI did not expose the cleanup-test-failover command.

Team Members

2400030408 G Vishnu Team Lead

2400030412 G Praneeth

2400030233 Ch Mohan Krishna

2400030418 K Yeswanth

1. Project Objective

The project demonstrates disaster recovery for a three-tier SpendWise application using Azure Site Recovery (ASR). The production workload consists of a database tier, application tier and web tier running as separate Azure VMs in Central India. ASR continuously replicates the protected workload toward India South Central so that the environment can be recovered after a regional or infrastructure failure.

1.1 Three-tier design

Tier	Production VM	DR/Test VM	Observed role
Database	VM-DB	VM-DB-test	PostgreSQL / database
Application	VM-APP	VM-APP-test	SpendWise Flask + Unicorn
Web	VM-WEB	VM-WEB-test	Web/application-facing tier

1.2 Architecture concept

The important flow demonstrated in the test environment was: Web → Application → Database. The application test VM successfully reached PostgreSQL on VM-DB-test at TCP/5432, and the web test VM successfully reached the application VM on TCP/5000. The application health endpoint returned HTTP 200 with database status connected.

- Production region: Central India.
- Recovery region: India South Central.
- ASR replication provider: A2A (Azure-to-Azure).
- Recovery/test network: VNET-ASR-TEST, subnet SUBNET-TEST.
- Recovery plan: RP-3TIER-APP.

2. Concepts Used

Azure Site Recovery (ASR)

Azure disaster recovery service that coordinates replication and recovery of workloads between a primary location and a recovery location.

Recovery Services Vault

Azure resource that stores ASR configuration, policies, replication metadata, recovery plans and related disaster-recovery management state.

A2A Replication

Azure-to-Azure replication. The production VM disks are replicated to recovery-side replica disks.

Fabric

ASR representation of a physical/cloud location. Here the primary fabric represents Central India and the recovery fabric represents India South Central.

Protection Container

Logical ASR container within a fabric that holds protected items. The source container used was PC-SOURCE-CENTRALINDIA.

Protected Item

ASR representation of a protected VM and its replicated disks/configuration. The three protected items were PI-VM-DB, PI-VM-APP and PI-VM-WEB.

Replication Policy

Defines replication behavior such as recovery-point/retention characteristics. The policy used was POLICY-A2A-SPENDWISE.

Recovery Plan

Defines an ordered recovery workflow for multiple protected VMs. RP-3TIER-APP contains the three application tiers.

Test Failover

Creates test recovery VMs from replicated data in an isolated/test network without intentionally performing a production failover.

Recovery Point

A consistent point from replicated data from which a recovery/test VM can be created. The test used LatestProcessed.

Network Isolation

Test VMs were placed in VNET-ASR-TEST so validation could be performed without putting test machines directly into the production network.

Run Command

Azure VM Run Command executes scripts inside a VM through Azure management services, useful for validating OS, services, ports and application behavior.

3. Environment and Naming

Resource	Value
Resource Group	RG-ASR-24CC3046
Vault	RSV-ASR-24CC3046
Primary Fabric	FABRIC-CENTRALINDIA
Primary Container	PC-SOURCE-CENTRALINDIA
Recovery Fabric	FABRIC-DR-ISC
Recovery Container	PC-DR-ISC
Replication Policy	POLICY-A2A-SPENDWISE
Recovery Plan	RP-3TIER-APP
Test VNet	VNET-ASR-TEST
Test subnet	SUBNET-TEST
DR Resource Group	RG-ASR-24CC3046-DR

4. Initial ASR Protection Verification

Before moving to recovery-plan testing, the protected items were checked. During the work, individual states progressed through:

- InitialReplicationInProgress
- InitialReplicationRecoveryCompletionPending
- InitialReplicationCompleted
- Protected

The final verification returned all three production VMs as Protected.

```
az site-recovery protected-item list -g RG-ASR-24CC3046 --vault-name RSV-ASR-24CC3046 \
  --fabric-name FABRIC-CENTRALINDIA \
  --protection-container PC-SOURCE-CENTRALINDIA \
  --query
"[].{VM:properties.friendlyName,State:properties.protectionState,Health:properties.replicationHealth}"
-o table
```

VM	State	Health
VM-WEB	Protected	Normal
VM-APP	Protected	Normal
VM-DB	Protected	Normal

Meaning: all three production VMs were under ASR protection and ASR reported Normal replication health.

5. Linux Guest Preparation and Kernel Verification

VM-WEB was checked using Azure Run Command. The original running kernel was 6.8.0-1064-azure. A 5.15 Azure kernel was installed and the VM was subsequently verified as running 5.15.0-1003-azure.

```
az vm run-command invoke -g RG-ASR-24CC3046 -n VM-WEB \
  --command-id RunShellScript --scripts "uname -r"

6.8.0-1064-azure

az vm run-command invoke -g RG-ASR-24CC3046 -n VM-WEB \
  --command-id RunShellScript --scripts "sudo apt-get update && sudo apt-get install -y linux-image-5.15.0-1003-azure"

Key installation output:
The following NEW packages will be installed:
  linux-image-5.15.0-1003-azure linux-modules-5.15.0-1003-azure
Fetched 30.3 MB in 1s
Setting up linux-image-5.15.0-1003-azure
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-5.15.0-1003-azure
update-initramfs: Generating /boot/initrd.img-5.15.0-1003-azure

az vm run-command invoke -g RG-ASR-24CC3046 -n VM-WEB \
  --command-id RunShellScript --scripts "uname -r"

5.15.0-1003-azure
```

Concept: the kernel change was verified inside the guest rather than inferred from package installation output. Installing a kernel does not itself prove that the machine booted into that kernel; the second uname command provides that proof.

6. OS Disk Identification

```
az vm show -g RG-ASR-24CC3046 -n VM-WEB \
  --query "storageProfile.osDisk.managedDisk.id" -o tsv

/subscriptions/b5f49426-f4c7-437e-97fc-c29a4ebd1430/resourceGroups/RG-ASR-24CC3046/providers/Microsoft.Compute/disks/VM-WEB_OsDisk_1_c9fe61099d67475d825aab735da1e4e3
```

Concept: a managed disk is the persistent storage backing the VM. ASR replicates protected disk state to a recovery-side replica disk.

7. Protecting VM-WEB with ASR

VM-WEB was registered as an ASR protected item using the A2A replication policy and provider-specific configuration.

```
az site-recovery protected-item create -g RG-ASR-24CC3046 \
  --vault-name RSV-ASR-24CC3046 \
  --fabric-name FABRIC-CENTRALINDIA \
  --name PI-VM-WEB \
  --protection-container PC-SOURCE-CENTRALINDIA \
  --policy-id "/subscriptions/b5f49426-f4c7-437e-97fc-c29a4ebd1430/resourceGroups/RG-ASR-24CC3046/providers/Microsoft.RecoveryServices/vaults/RSV-ASR-24CC3046/replicationPolicies/POLICY-A2A-SPENDWISE" \
  --provider-details "$(cat /tmp/vm-web-details.json)"
```

Key returned state:

```
"protectionState": "InitialReplicationInProgress",
"protectionStateDescription": "Initial replication is in progress",
"replicationHealth": "Normal",
"primaryFabricFriendlyName": "Central India",
"recoveryFabricLocation": "indiasouthcentral",
"osType": "Linux",
"protectedManagedDisks": [
  {
    "diskState": "InitialReplicationPending",
    "diskType": "OperatingSystem",
    "recoveryReplicaDiskAccountType": "Premium_LRS",
    "recoveryResourceGroupId": ".../RG-ASR-24CC3046-DR"
  }
]
```

Concept: InitialReplicationInProgress means ASR is creating/synchronizing the initial replica. The protected item is not considered fully protected until the initial synchronization completes and the state reaches Protected.

8. Recovery Plan Creation

The first recovery-plan command failed because the installed Azure CLI required explicit --groups, --primary-fabric-id and --recovery-fabric-id arguments. The command was then corrected using full fabric resource IDs.

```
az site-recovery recovery-plan create -g RG-ASR-24CC3046 \
  --vault-name RSV-ASR-24CC3046 \
  --name RP-3TIER-APP \
  --source-fabric FABRIC-CENTRALINDIA \
  --target-fabric FABRIC-DR-ISC
```

Error:

the following arguments are required:

--groups, --primary-fabric-id, --recovery-fabric-id

Corrected creation used three Boot groups containing DB, APP and WEB.

```
az site-recovery recovery-plan create -g RG-ASR-24CC3046 \
  --vault-name RSV-ASR-24CC3046 \
  --name RP-3TIER-APP \
  --primary-fabric-id "$PRIMARY" \
  --recovery-fabric-id "$RECOVERY" \
```

```
--failover-deployment-model ResourceManager \
--groups '<groups JSON containing PI-VM-DB, PI-VM-APP and PI-VM-WEB>'
```

Key returned values:

```
"name": "RP-3TIER-APP",
"primaryFabricFriendlyName": "Central India",
"recoveryFabricFriendlyName": "India South Central",
"replicationProviders": ["A2A"]
```

Boot groups:

1. PI-VM-DB
2. PI-VM-APP
3. PI-VM-WEB

8.1 Recovery plan verification

```
az site-recovery recovery-plan show -g RG-ASR-24CC3046 \
--vault-name RSV-ASR-24CC3046 --name RP-3TIER-APP \
--query "properties.groups[...].{Type:groupType,VMs:replicationProtectedItems[...].id}" -o json
```

```
[
  {"Type":"Shutdown","VMs":[]},
  {"Type":"Failover","VMs":[]},
  {"Type":"Boot","VMs":["../PI-VM-DB"]},
  {"Type":"Boot","VMs":["../PI-VM-APP"]},
  {"Type":"Boot","VMs":["../PI-VM-WEB"]}
]
```

Concept: a recovery plan groups multiple protected items into an orchestrated recovery workflow. This is particularly useful for multi-tier applications because services have dependencies and should be recovered as a coordinated workload.

9. Test Failover Network

```
az network vnet show -g RG-ASR-24CC3046 -n VNET-ASR-TEST --query id -o tsv
```

```
/subscriptions/b5f49426-f4c7-437e-97fc-c29a4ebd1430/resourceGroups/RG-ASR-
24CC3046/providers/Microsoft.Network/virtualNetworks/VNET-ASR-TEST
```

The test network was used to keep test-failover VMs isolated from the production network.

10. Test Failover Execution

The installed Azure CLI did not expose a recovery-plan test-failover subcommand, so the recovery-plan REST endpoint was used.

```
az site-recovery recovery-plan test-failover -h
```

'test-failover' is misspelled or not recognized by the system.

A request body was prepared with PrimaryToRecovery, the test VNet and LatestProcessed recovery point.

```
{
  "properties": {
    "failoverDirection": "PrimaryToRecovery",
    "networkId": "/subscriptions/.../resourceGroups/RG-ASR-
24CC3046/providers/Microsoft.Network/virtualNetworks/VNET-ASR-TEST",
    "networkType": "VmNetworkAsInput",
    "providerSpecificDetails": [
      {
        "instanceType": "A2A",
        "recoveryPointType": "LatestProcessed"
```

```

    }
  ]
}
}

```

```

az rest --method post --url "https://management.azure.com/subscriptions/.../resourceGroups/RG-ASR-24CC3046/providers/Microsoft.RecoveryServices/vaults/RSV-ASR-24CC3046/replicationRecoveryPlans/RP-3TIER-APP/testFailover?api-version=2025-02-01" --body @/tmp/test-failover.json

```

The returned job was monitored through the ASR job list.

```

az site-recovery job list -g RG-ASR-24CC3046 \
  --vault-name RSV-ASR-24CC3046 \
  --query "[?contains(properties.scenarioName, 'TestFailover')] | contains(properties.scenarioName, 'testFailover')].[Name:name,State:properties.state,Scenario:properties.scenarioName]" -o table

```

Name	State	Scenario
7df5bc18-4a2e-40ff-83da-1b21f16ea647	InProgress	TestFailover

```

az site-recovery job show -g RG-ASR-24CC3046 \
  --vault-name RSV-ASR-24CC3046 \
  --job-name 7df5bc18-4a2e-40ff-83da-1b21f16ea647 \
  --query "{State:properties.state,Start:properties.startTime,End:properties.endTime,Error:properties.error}" -o json
{
  "End": "2026-09-26T22:09:43Z",
  "Error": null,
  "Start": "2026-09-26T22:04:19.3649464Z",
  "State": "Succeeded"
}

```

Measured execution duration: approximately 5 minutes 24 seconds from 22:04:19 to 22:09:43 UTC.

11. Verify Test-Failover VMs

```

az vm list -g RG-ASR-24CC3046-DR --query "[].{Name:name,State:powerState,PrivateIP:networkProfile.networkInterfaces[0].id}" -o table

```

Name	PrivateIP
VM-APP-test	.../networkInterfaces/VM-APPVMNic-test
VM-DB-test	.../networkInterfaces/VM-DBVMNic-test
VM-WEB-test	.../networkInterfaces/VM-WEBVMNic-test

11.1 Verify subnet placement

```

az network nic show -g RG-ASR-24CC3046-DR -n VM-DBVMNic-test \
  --query "ipConfigurations[0].subnet.id" -o tsv
.../virtualNetworks/VNET-ASR-TEST/subnets/SUBNET-TEST

az network nic show -g RG-ASR-24CC3046-DR -n VM-APPVMNic-test \
  --query "ipConfigurations[0].subnet.id" -o tsv
.../virtualNetworks/VNET-ASR-TEST/subnets/SUBNET-TEST

az network nic show -g RG-ASR-24CC3046-DR -n VM-WEBVMNic-test \
  --query "ipConfigurations[0].subnet.id" -o tsv
.../virtualNetworks/VNET-ASR-TEST/subnets/SUBNET-TEST

```

11.2 Verify private IP addresses

```
az vm list-ip-addresses -g RG-ASR-24CC3046-DR -n VM-DB-test \
  --query "[0].virtualMachine.network.privateIpAddresses[0]" -o tsv
10.30.1.4

az vm list-ip-addresses -g RG-ASR-24CC3046-DR -n VM-APP-test \
  --query "[0].virtualMachine.network.privateIpAddresses[0]" -o tsv
10.30.1.5

az vm list-ip-addresses -g RG-ASR-24CC3046-DR -n VM-WEB-test \
  --query "[0].virtualMachine.network.privateIpAddresses[0]" -o tsv
10.30.1.6
```

12. Three-Tier Application Validation

This was the most important functional validation because successful VM recovery alone does not prove that the application works.

12.1 APP → DB connectivity

```
az vm run-command invoke -g RG-ASR-24CC3046-DR -n VM-APP-test \
  --command-id RunShellScript \
  --scripts "timeout 5 bash -c '</dev/tcp/10.30.1.4/5432' && echo DB_PORT_5432_OK || echo
DB_PORT_5432_FAILED"

DB_PORT_5432_OK
```

Meaning: the recovered application VM could establish a TCP connection to PostgreSQL on the recovered database VM.

12.2 WEB → APP TCP connectivity

```
az vm run-command invoke -g RG-ASR-24CC3046-DR -n VM-WEB-test \
  --command-id RunShellScript \
  --scripts "timeout 5 bash -c '</dev/tcp/10.30.1.5/5000' && echo APP_PORT_5000_OK || echo
APP_PORT_5000_FAILED"

APP_PORT_5000_OK
```

12.3 First application health test

```
az vm run-command invoke -g RG-ASR-24CC3046-DR -n VM-WEB-test \
  --command-id RunShellScript \
  --scripts "curl -s -o /tmp/health.txt -w 'HTTP_STATUS=%{http_code}\n' http://10.30.1.5:5000/health;
cat /tmp/health.txt"

HTTP_STATUS=000
cat: /tmp/health.txt: No such file or directory
```

This indicated that although TCP/5000 was reachable, the HTTP request did not receive a response. The next diagnostic step was therefore performed on the application VM itself.

12.4 Diagnose application service

```
az vm run-command invoke -g RG-ASR-24CC3046-DR -n VM-APP-test \
  --command-id RunShellScript \
  --scripts "sudo systemctl status spendwise --no-pager; echo PORT; sudo ss -lntp | grep 5000 || true"

spendwise.service - SpendWise Flask Application
Active: active (running)
Main PID: 939 (gunicorn)
Listening at: http://0.0.0.0:5000
```

```
PORT
LISTEN 0 2048 0.0.0.0:5000 0.0.0.0:* users:(("unicorn",...))
```

```
A worker timeout was also observed:
[CRITICAL] WORKER TIMEOUT
[ERROR] Worker ... was sent SIGKILL! Perhaps out of memory?
```

Important concept: listening on a port proves that a process has bound the socket; it does not guarantee that an HTTP endpoint can complete successfully. The health endpoint performs a database connection, so database configuration also had to be validated.

13. Application Configuration Correction

The recovered app.py contained the production database IP 10.10.3.4. The recovered database VM was at 10.30.1.4.

```
cd /opt/spendwise && sed -n '1,240p' app.py

DB_HOST = "10.10.3.4"
DB_NAME = "spendwise"
DB_USER = "spenduser"
DB_PASSWORD = "Spend@12345"
...
@app.route("/health")
def health():
    try:
        conn = get_db_connection()
        conn.close()
        return jsonify({
            "application": "SpendWise",
            "database": "connected",
            "status": "healthy"
        })
    ...

sudo sed -i 's/DB_HOST = "10.10.3.4"/DB_HOST = "10.30.1.4"/' /opt/spendwise/app.py \
    && sudo systemctl restart spendwise \
    && grep DB_HOST /opt/spendwise/app.py

DB_HOST = "10.30.1.4"
host=DB_HOST,
```

13.1 Database access-control correction

```
echo 'host spendwise spenduser 10.30.1.0/24 scram-sha-256' | \
    sudo tee -a /etc/postgresql/14/main/pg_hba.conf \
    && sudo systemctl reload postgresql \
    && sudo tail -n 5 /etc/postgresql/14/main/pg_hba.conf

host spendwise spenduser 10.30.1.0/24 scram-sha-256
local replication all peer
host replication all 127.0.0.1/32 scram-sha-256
host replication all ::1/128 scram-sha-256
host spendwise spenduser 10.10.2.0/24 scram-sha-256
host spendwise spenduser 10.30.1.0/24 scram-sha-256
```

Concept: pg_hba.conf controls which clients may authenticate to PostgreSQL and which authentication method is required. The first application health test failed with 'no pg_hba.conf entry for host 10.30.1.5'. Adding the recovery subnet and reloading PostgreSQL allowed the recovered application VM to authenticate.

14. Final End-to-End Validation

```
az vm run-command invoke -g RG-ASR-24CC3046-DR -n VM-APP-test \
--command-id RunShellScript \
--scripts "curl -sS --max-time 10 -w '\nHTTP_STATUS=%{http_code}\n' http://127.0.0.1:5000/health"

{"application":"SpendWise","database":"connected","status":"healthy"}

HTTP_STATUS=200

az vm run-command invoke -g RG-ASR-24CC3046-DR -n VM-WEB-test \
--command-id RunShellScript \
--scripts "curl -sS --max-time 10 -w '\nHTTP_STATUS=%{http_code}\n' http://10.30.1.5:5000/health"

{"application":"SpendWise","database":"connected","status":"healthy"}

HTTP_STATUS=200
```

Final meaning: the recovered application tier could reach the recovered database tier, and the recovered web tier could call the recovered application tier. The application health endpoint also confirmed database connectivity and returned HTTP 200.

15. Test Failover Cleanup Status

A cleanup command was attempted after validation:

```
az site-recovery recovery-plan cleanup-test-failover \
-g RG-ASR-24CC3046 \
--vault-name RSV-ASR-24CC3046 \
--name RP-3TIER-APP \
--comments "Completed ASR test failover and three-tier application validation"

'cleanup-test-failover' is misspelled or not recognized by the system.
```

Review status: Do not claim test-failover cleanup is complete based on this output. The test failover and validation are complete; cleanup remains a separate pending administrative step unless performed through a supported portal/API operation.

This distinction is important in a technical review: a successful TestFailover job proves the recovery test worked; it does not by itself prove that the temporary test resources were subsequently removed.

16. Evidence Summary for Review

Checkpoint	Actual evidence	Conclusion
Replication	VM-WEB, VM-APP, VM-DB = Protected; Health = Normal	Production workload protected
Recovery plan	RP-3TIER-APP contains DB, APP and WEB Boot groups	Three-tier orchestration configured
Test failover	Job 7df5... State = Succeeded; Error = null	ASR test recovery succeeded
Recovery VMs	VM-DB-test, VM-APP-test, VM-WEB-test created	Test workload instantiated
Network	All test NICs on VNET-ASR-TEST/SUBNET-TEST	Test workload isolated
APP→DB	DB_PORT_5432_OK	Database network path works
WEB→APP	APP_PORT_5000_OK	Application network path works
Application	HTTP_STATUS=200; database=connected; status=healthy	Three-tier application is functionally validated
Cleanup	CLI command not recognized	Cleanup must not be claimed as

17. Review-Day Command Cheat Sheet

Run these in this order when demonstrating the project.

1. Protected items

```
az site-recovery protected-item list -g RG-ASR-24CC3046 --vault-name RSV-ASR-24CC3046 --fabric-name FABRIC-CENTRALINDIA --protection-container PC-SOURCE-CENTRALINDIA --query "[].{VM:properties.friendlyName,State:properties.protectionState,Health:properties.replicationHealth}" -o table
```

2. Recovery plan

```
az site-recovery recovery-plan show -g RG-ASR-24CC3046 --vault-name RSV-ASR-24CC3046 --name RP-3TIER-APP --query "properties.groups[0].{Type:groupType,VMs:replicationProtectedItems[0].id}" -o json
```

3. Latest test-failover job

```
az site-recovery job list -g RG-ASR-24CC3046 --vault-name RSV-ASR-24CC3046 --query "[?properties.scenarioName=='TestFailover'] | sort_by(@,&properties.startTime) | [-1].{Name:name,Scenario:properties.scenarioName,State:properties.state,Start:properties.startTime,End:properties.endTime,Error:properties.error}" -o table
```

4. Test VMs

```
az vm list -g RG-ASR-24CC3046-DR -o table
```

5. APP→DB

```
az vm run-command invoke -g RG-ASR-24CC3046-DR -n VM-APP-test --command-id RunShellScript --scripts "timeout 5 bash -c '</dev/tcp/10.30.1.4/5432' && echo DB_PORT_5432_OK || echo DB_PORT_5432_FAILED"
```

6. WEB→APP

```
az vm run-command invoke -g RG-ASR-24CC3046-DR -n VM-WEB-test --command-id RunShellScript --scripts "timeout 5 bash -c '</dev/tcp/10.30.1.5/5000' && echo APP_PORT_5000_OK || echo APP_PORT_5000_FAILED"
```

7. Final health

```
az vm run-command invoke -g RG-ASR-24CC3046-DR -n VM-WEB-test --command-id RunShellScript --scripts "curl -sS --max-time 10 -w '\nHTTP_STATUS=%{http_code}\n' http://10.30.1.5:5000/health"
```

18. Final Project Status

Completed: ASR protection of VM-WEB, VM-APP and VM-DB; recovery plan creation; test failover; creation of recovered test VMs; test-network placement; DB connectivity; APP connectivity; application health validation.

Key measured result: TestFailover job succeeded from 2026-09-26 22:04:19 UTC to 22:09:43 UTC, approximately 5 minutes 24 seconds.

Functional result: SpendWise health endpoint returned HTTP 200 and reported database connected.

Pending / do not overclaim: Test-failover cleanup was not confirmed because the attempted CLI cleanup command was unavailable.

For the project review, the strongest story is not simply 'ASR created VMs'. The stronger technical story is: production VMs were protected → recovery plan orchestrated the tiers → test failover created isolated DR VMs →

networking was verified → the application was diagnosed and reconfigured for the recovered database → PostgreSQL access control was corrected → the complete application path returned HTTP 200.